Reflection

Ali Abbas (24968341)
Advanced Algorithms Assignment 2

Jarvis March Reflection

This algorithm was simple in its essence. I'd say the most complex part was understanding that, instead of sorting like Graham Scan, it scans every point from the current vertex, uses the cross product to compare candidates, and keeps the one with no points beyond it. What surprised me was how fragile that simple loop is. On our nearly collinear dataset, floating-point rounding made the comparisons inconsistent, so the wrap circled back to a vertex that wasn't the start and never terminated. Our test suite caught it, and the fix was a small safety check that stops if the next point is already on the hull. It showed me that even simple algorithms need care in the edge cases.

If I implemented it again, I would separate the three steps clearly before writing any code: choosing the starting point, finding the next vertex, and detecting the return to the start. I would also write tests for duplicates, collinear inputs and very small inputs from the beginning, rather than adding them after something broke.

Chan's Algorithm Reflection

Chan's Algorithm runs in O(n log h) and combines concepts from both of the other algorithms we implemented. It was definitely the harder of the two. Each idea is simple on its own: split the points into groups of size m, build each group's hull with a sorting-based scan, then gift-wrap over those small hulls. The difficulty is in how they fit together. The hardest part for me was the tangent search, which uses binary search to find the most clockwise vertex of each group's hull from the current point. Without it, the wrapping step falls back to checking every point, and the algorithm loses the O(n log h) bound that makes it worth using. The other subtle piece was the guessing strategy: squaring m each round (4, 16, 256) so that the failed attempts only add a constant factor.

Numerical robustness came up again here. On the collinear data, rounding broke the assumptions behind the binary search, so I added a local check with a linear-scan fallback, which was needed 391 times at n = 10,000. That made Chan's slower on collinear data than on random data of the same size.

The biggest lesson came from the benchmarks. On paper Chan's should never lose, but it was never the fastest in our controlled experiment at n = 100,000: Jarvis won on small hulls and Graham Scan on large ones. Its grouping, repeated sorting and tangent searches cost more than its better bound saves at these sizes. If I did it again, I would profile it early to see where that overhead goes, and test on much larger inputs to find where its advantage actually appears.